

Contents

□ Hashing Patterns — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	3
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	4
SECTION 6 — Time & Space Complexity	5
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	7
SECTION 9 — Real World Applications	7
SECTION 10 — Interview Strategy	7
SECTION 11 — Pattern Variations	8
SECTION 12 — Comparison With Other Patterns	8
SECTION 13 — Problem Classification	9
SECTION 14 — 30 Interview Problems	9
SECTION 15 — Common Mistakes	12
SECTION 16 — Optimization Techniques	12
SECTION 17 — Multiple Language Templates	12
SECTION 18 — Dry Runs	14
SECTION 19 — Advanced Concepts	14
SECTION 20 — Senior Engineer Insights	14
SECTION 21 — Cheat Sheet	15
SECTION 22 — Revision Notes (5-Minute Review)	15
SECTION 23 — Practice Roadmap	15
SECTION 24 — Memory Tricks	16
SECTION 25 — Final Summary	16

□ Hashing Patterns — Complete Interview Handbook

Pattern #9 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Hashing section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Hashing uses a **hash function** to map keys to array indices (buckets) in expected $O(1)$ time, enabling near-constant-time insertion, lookup, and deletion regardless of data order. In interviews, “Hashing Patterns” refers to using hash maps/sets to track **frequency counts, seen-elements, complements, or groupings** — trading $O(n)$ space for eliminating the need for sorting or nested loops.

1.2 Why Was This Pattern Invented?

Arrays give $O(1)$ access only by *index*; hash tables generalize this to $O(1)$ *expected* access by *arbitrary key* (via a hash function converting the key into an index). This solves the fundamental problem: “how do I check membership, count frequency, or find complements without scanning the whole dataset repeatedly?”

1.3 Real Intuition Behind The Pattern

A hash map is like a **library’s card catalog system with numbered drawers** — instead of searching every shelf for a book ($O(n)$ linear search), you compute which drawer number the book’s title hashes to, and go directly there ($O(1)$ expected).

1.4 Mental Model

“Have I seen this before?” and “What’s missing to complete this?” are the two dominant hashing questions. A hash set answers “have I seen this,” a hash map answers “how many times / what value is associated,” and both let you check “does the complement I need already exist” in $O(1)$ instead of re-scanning.

1.5 Visual Explanation

Two Sum (unsorted array, need $O(n)$ not $O(n^2)$):

arr = [2, 7, 11, 15], target = 9

seen = {}

i=0: need $9-2=7$. Is 7 in seen? No. Add 2 → seen={2:0}

i=1: need $9-7=2$. Is 2 in seen? YES (index 0) → return (0,1)

1.6 Simple Analogy

Hashing is like a **coat-check counter** — you hand over your coat (key) and get a ticket (hash), and retrieving it later is a direct lookup by ticket number, not searching through every coat on the rack.

1.7 When Should I Immediately Think About Using This Pattern?

- Need to check **existence/membership** quickly (“have I seen X before?”).
 - Need **frequency counts** (anagrams, majority element, most frequent).
 - Need to find a **complement** (Two Sum, pair/triplet sums) on **unsorted** data.
 - Need to **group** items by a computed key (Group Anagrams, group by remainder).
 - The array is **unsorted** and sorting would be too slow or would destroy needed index information.
-

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
"have you seen," "does there exist"	Hash set membership check
"frequency," "count of," "most/least frequent"	Hash map frequency counting
"two sum," "pair that sums to" (unsorted)	Complement lookup via hash map
"anagram," "group by"	Hash map keyed by sorted string / frequency signature
"duplicate," "unique"	Hash set membership
"first non-repeating"	Hash map with order tracking

2.2 Hidden Hints

Array explicitly stated as **unsorted**, combined with an $O(n)$ or $O(n \log n)$ expected complexity for a pair/complement search — this combination almost always signals hashing over Two Pointers (which needs sortedness).

2.3 Interview Clues

Interviewer says "the array is not sorted" right after you propose a Two Pointers approach — direct signal to pivot to hashing.

2.4 Common Trick Words

"Complement," "signature," "canonical form" (for anagram grouping via sorted-string keys), "prefix" (when combined with Prefix Sum for subarray-sum counting).

2.5 What Interviewers Expect

Correct choice between hash set (existence only) vs hash map (existence + associated data/count), awareness of $O(n)$ space trade-off, and discussion of hash collision handling / amortized $O(1)$ vs worst-case $O(n)$ lookup in adversarial cases.

2.6 When NOT To Use This Pattern

- **Order matters** and you need range queries (min/max/sorted order) — hash tables don't maintain order; use a sorted structure (TreeMap/balanced BST) or sorting instead.
- **Memory is severely constrained** and $O(n)$ extra space isn't acceptable — consider Two Pointers (if sortable) or Cyclic Sort (if bounded range) instead.
- You need to preserve **insertion order with range-based queries** — a plain hash map doesn't support "give me all keys between X and Y" efficiently.

SECTION 3 — Decision Framework

Do you need to check EXISTENCE or COUNT FREQUENCY of elements?

Yes

Is the array UNSORTED, or does sorting destroy needed information (indices/order)?

Yes → USE HASHING ($O(n)$ time, $O(n)$ space)

No (array is sorted or sortable without losing info)

Consider TWO POINTERS instead ($O(1)$ space) if a pair/triplet search is the goal

Do you need MIN/MAX or RANGE queries over the keys (not just existence/count)?

Yes → Hashing alone is insufficient - use a sorted structure (TreeMap, sorted array + binary search)

Is space EXTREMELY constrained ($O(1)$ required)?

Yes → Consider Cyclic Sort (if bounded range) or Two Pointers (if sortable) instead of Hashing

Why: Hashing's core trade is $O(n)$ space for $O(1)$ expected time existence/frequency checks, regardless of data order — this makes it strictly more broadly applicable than Two Pointers or Cyclic Sort, but at the cost of extra memory and the loss of any ordering guarantees.

SECTION 4 — Why This Pattern Works

Mathematical: A well-designed hash function distributes n keys roughly uniformly across m buckets. With load factor $= n/m$ kept low (typically via dynamic resizing), the expected number of keys per bucket is $O(1)$, so insertion/lookup/deletion are $O(1)$ **expected** time (not worst-case — a pathological hash function or adversarial input could cause $O(n)$ worst case, mitigated in practice by randomized hashing/salting).

Logical: Checking “does the complement exist” via a hash map lookup is $O(1)$ expected, versus $O(n)$ for a linear scan or $O(\log n)$ for a sorted-array binary search — over n elements, this turns $O(n^2)$ brute-force pair-checking into $O(n)$ total.

Intuitive: Instead of asking “is X anywhere in this list” by looking at every element, you ask “what bucket would X be in if it existed” and check only that one spot.

Correctness Proof (Two Sum via hashing): *Invariant:* at each step i , the hash map contains exactly the elements $\text{arr}[0..i-1]$. *Base case:* before processing any element, the map is empty — trivially correct. *Inductive step:* at step i , checking $\text{target} - \text{arr}[i]$ against the map correctly determines whether some earlier element completes the pair, since the map faithfully represents all previously seen elements; then $\text{arr}[i]$ is added, extending the invariant. *Termination:* after processing all n elements, either a valid pair was found and returned, or none exists among any prefix pairing, hence none exists at all. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (Complement Lookup — Two Sum style)

1. Initialize an empty hash map `seen`.
2. For each element x at index i : compute $\text{complement} = \text{target} - x$.
3. If `complement` is in `seen`, return $(\text{seen}[\text{complement}], i)$.
4. Else, add x : i to `seen`.
5. Continue until found or array exhausted.

5.2 Step-by-Step Framework (Frequency Counting)

1. Initialize an empty hash map `freq`.
2. For each element, increment `freq[element]`.

3. Use freq to answer questions (most frequent, count of distinct, anagram check via comparing frequency maps).

5.3 General Template — Complement Lookup

```
function twoSum(arr, target):
    seen = {}
    for i in range(0, length(arr)):
        complement = target - arr[i]
        if complement in seen:
            return [seen[complement], i]
        seen[arr[i]] = i
    return [-1, -1]
```

5.4 General Template — Frequency Counting / Grouping

```
function groupAnagrams(strings):
    groups = {}
    for s in strings:
        key = sorted(s)                # canonical signature
        if key not in groups:
            groups[key] = []
        groups[key].append(s)
    return values(groups)
```

5.5 Interview Thinking Process

1. “This needs $O(1)$ existence/frequency checks over unsorted data — I’ll use a hash map/set.”
 2. “I’ll decide: do I need just membership (hash set) or an associated value/count (hash map)?”
 3. “I’ll process elements in a single pass, checking the map before inserting the current element (to correctly handle the ‘find a pair’ case without matching an element with itself).”
 4. “I’ll mention the $O(n)$ space trade-off versus a sorted/Two-Pointers approach explicitly.”
-

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(n)$ expected; $O(n^2)$ pathological (rare, mitigated by good hash functions/resizing)	$O(n)$	Each of n elements processed once; hash collisions bounded in practice
Average Case	$O(n)$	$O(n)$	Uniform hash distribution keeps buckets small
Best Case	$O(n)$ (must still scan once)	$O(n)$	Even trivial cases require building the structure

Case	Time	Space	Why
Amortized	$O(1)$ per operation amortized across dynamic resizing	$O(n)$	Resizing (doubling) amortizes the occasional $O(n)$ rehash cost across many $O(1)$ operations

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty array	[]	Return immediately, no pairs/frequencies to compute
Single element	[5]	No pair possible with itself unless problem explicitly allows self-pairing
All duplicate elements	[3, 3, 3, 3]	Frequency map correctly counts all occurrences; pair-finding must avoid matching an element with itself unless index differs
Negative numbers	[-3, 4, -1]	Hashing handles negatives natively — no special casing needed
Target achieved by same element twice (self-pairing)	arr=[5], target=10	Must check complement $\neq$ arr[i] unless duplicate values with different indices exist
Very large key space (strings, tuples)	Complex objects as keys	Requires well-defined equality/hashCode implementation (language-dependent)
Hash collisions (adversarial input)	Many keys hashing to the same bucket	Rare in interviews, but worth mentioning; production hash tables use randomized seeds to mitigate
Case sensitivity in string keys	"Eat" vs "eat"	Clarify whether normalization (lowercasing) is expected before hashing

Common mistakes: checking the map for the complement **after** inserting the current element instead of before, causing incorrect self-pairing on some inputs; forgetting to normalize keys (case, whitespace) when required by the problem.

SECTION 8 — Pros & Cons

Advantages: $O(1)$ expected time for existence/frequency/lookup operations; works on unsorted data with no preprocessing sort required; highly flexible (any hashable key type). **Disadvantages:** $O(n)$ extra space; no ordering guarantees; worst-case $O(n)$ per operation under pathological hash collisions (rare but theoretically possible). **Trade-offs:** Hashing ($O(n)$ time, $O(n)$ space, no order needed) vs. Two Pointers ($O(n)$ time post-sort, $O(1)$ space, requires sortedness) vs. sorting + binary search ($O(n \log n)$ time, $O(1)$ extra space, gives ordering). **Limitations:** Poor fit for range queries (min/max/between); not ideal when memory is severely constrained. **Inefficient when:** the key space is enormous and sparse with a poor hash function (excessive collisions), or when order-dependent operations dominate the workload.

SECTION 9 — Real World Applications

Domain	Application
Google	Search index deduplication — hash-based fingerprinting to detect duplicate web pages at scale
Amazon	Product catalog deduplication, session/cart lookups via hashed customer/session IDs
Meta	News Feed deduplication of seen content IDs per user session (hash set of recently shown post IDs)
Netflix	Content recommendation caching — hashed user-item interaction lookups
Databases	Hash indexes for equality lookups (as opposed to B-Tree indexes for range queries)
Distributed Systems	Consistent hashing for load balancing and sharding across servers
Networking	Hash-based routing table lookups (e.g., ECMP hashing for load distribution)
Security	Password hashing (though cryptographic hashing has different properties than data-structure hashing), hash-based message authentication codes (HMAC)
Blockchain	Hash-based Merkle trees for efficient data integrity verification
Compilers	Symbol tables in compilers are hash maps mapping identifiers to their metadata

SECTION 10 — Interview Strategy

How seniors answer: They immediately identify whether the problem needs membership (set) or associated data/counts (map), state the $O(n)$ space trade-off versus alternatives (Two Pointers, sorting) explicitly, and correctly sequence the “check before insert” logic to avoid self-pairing bugs.

How juniors answer: They often default to nested loops ($O(n^2)$) without considering hashing, or they insert into the map before checking for the complement, causing subtle self-matching bugs on certain inputs.

Typical follow-ups: “Can you do this with $O(1)$ space instead?” (discuss Two Pointers if sortable, or Cyclic Sort if bounded range). “What if the data doesn’t fit in memory?” (discuss external hashing / streaming approaches like Bloom filters). “How do you handle hash collisions?” (discuss chaining vs open addressing, load factor, resizing).

Optimization questions: “Can you reduce space by using a bitset instead of a full hashmap for a bounded integer range?” (yes, when applicable — a nice space optimization to mention).

SECTION 11 — Pattern Variations

Variation	Description	Example
Existence Check (Hash Set)	Track “have I seen this”	Contains Duplicate
Complement Lookup (Hash Map)	Track value $\rightarrow$ index/count for pair-finding	Two Sum
Frequency Counting	Track occurrence counts	Top K Frequent Elements, Valid Anagram
Grouping by Signature	Bucket items by a computed canonical key	Group Anagrams
Prefix Sum + Hashing	Track cumulative sum frequencies for subarray-sum counting	Subarray Sum Equals K
Sliding Window + Hashing	Track window state via frequency map	Longest Substring Without Repeating Characters
Set-Based Union/Intersection	Compare membership across two collections	Intersection of Two Arrays

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Two Pointers	$O(1)$ space but requires sorted/sortable data	Data is sorted or sorting doesn’t lose needed info
Cyclic Sort	$O(1)$ space but requires bounded value range matching array size	Values bounded to exactly $1..n$ or $0..n-1$
Sorting + Binary Search	$O(1)$ extra space (in-place sort), $O(\log n)$ query, but $O(n \log n)$ upfront and loses original order unless tracked	Order/range queries matter, one-time preprocessing acceptable
Sliding Window	Uses hashing internally for window state, but the “window” structure itself is the primary technique	Contiguous subrange problems

Comparison Table

Aspect	Hashing	Two Pointers	Sorting + Binary Search
Requires sorted data	No	Yes	No (sorts internally)
Space	$O(n)$	$O(1)$	$O(1)$ extra (in-place)
Time	$O(n)$ expected	$O(n \log n)$ if sort needed, else $O(n)$	$O(n \log n)$
Preserves original order/index	Yes (naturally)	Only if tracked separately	Only if tracked separately

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Direct existence/frequency check	Two Sum, Contains Duplicate, Valid Anagram
Medium	Grouping, complement counting with additional constraints	Group Anagrams, Subarray Sum Equals K, Top K Frequent Elements
Hard	Combined with sliding window or prefix sum for advanced counting	Longest Substring Without Repeating Characters, Minimum Window Substring
Very Hard	Multi-hashmap combinations, advanced canonical key design	LRU Cache (hash + doubly linked list), Insert Delete GetRandom $O(1)$, Randomized Set design problems

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Two Sum	Easy	Amazon, Meta, Microsoft, Google, Apple	Complement lookup on unsorted array	Foundational hashing
2	Contains Duplicate	Easy	Amazon, Meta	Hash set existence check	Basic set usage
3	Valid Anagram	Easy	Amazon, Meta, Microsoft	Frequency map comparison	Frequency counting basics
4	Group Anagrams	Medium	Amazon, Meta, Microsoft, Google	Canonical signature grouping	Grouping by computed key

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
5	Top K Frequent Elements	Medium	Amazon, Meta, Google	Frequency map + bucket/heap selection	Frequency + selection combination
6	Subarray Sum Equals K	Medium	Amazon, Meta, Google, Microsoft	Prefix sum + hashmap frequency	Cross-pattern combination
7	Longest Consecutive Sequence	Medium	Amazon, Meta, Google, Microsoft	Hash set for O(n) sequence detection	Advanced set-based sequence finding
8	Longest Substring Without Repeating Characters	Medium	Amazon, Meta, Google	Sliding window + hash map	Cross-pattern (Window + Hashing)
9	Intersection of Two Arrays	Easy	Meta, Google	Set intersection	Basic set operations
10	Intersection of Two Arrays II	Easy	Meta, Google	Frequency map intersection	Frequency-aware set operations
11	Isomorphic Strings	Easy	Amazon, Meta	Bidirectional hash map mapping	Mapping consistency checking
12	Word Pattern	Easy	Google, Amazon	Bidirectional hash map mapping	Mapping consistency checking
13	Ransom Note	Easy	Amazon, Meta	Frequency map subtraction check	Basic frequency comparison
14	First Unique Character in a String	Easy	Amazon, Microsoft	Frequency map + order tracking	Order-aware frequency counting
15	4Sum II	Medium	Amazon, Meta, Google	Hashmap of pairwise sums	Advanced complement counting
16	Find All Anagrams in a String	Medium	Meta, Amazon	Sliding window + frequency map	Cross-pattern combination
17	LRU Cache	Medium	Amazon, Meta, Google, Microsoft	Hash map + doubly linked list	Advanced data structure design
18	LFU Cache	Hard	Amazon, Google	Multi-level hash map + frequency buckets	Advanced multi-hashmap design

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
19	Insert Delete GetRandom O(1)	Medium	Amazon, Meta	Hash map + array combination	O(1) randomized structure design
20	Design HashMap	Easy	Amazon, Google	Implementing hashing from scratch	Foundational hash table mechanics
21	Continuous Subarray Sum	Medium	Meta, Amazon	Prefix sum + modulo hashing	Modular hashing combination
22	Subarray Sums Divisible by K	Medium	Google, Amazon	Prefix sum + modulo bucket counting	Modulo counting variant
23	Determine if Two Strings Are Close	Medium	Google	Frequency signature comparison	Advanced frequency comparison
24	Two Sum II — Input Array Is Sorted (contrast)	Easy	Amazon	Contrast: Two Pointers preferred here	Pattern-boundary awareness
25	Brick Wall	Medium	Google, Amazon	Frequency map of cumulative edge positions	Creative hashing application
26	Find Duplicate Subtrees	Medium	Amazon, Meta	Hash map of serialized subtree signatures	Cross-pattern (Tree + Hashing)
27	Encode and Decode TinyURL	Medium	Amazon, Google	Hash map for URL-ID mapping	System-design-adjacent hashing
28	Random Pick with Weight	Medium	Amazon, Meta	Prefix sum + binary search (contrast, not pure hashing)	Pattern-boundary awareness
29	Design Twitter	Medium	Amazon, Meta, Google	Hash map + heap combination for feed generation	Advanced multi-structure combination
30	Employee Free Time (contrast)	Hard	Amazon, Google	Contrast: Merge Intervals, not hashing	Pattern-boundary awareness

SECTION 15 — Common Mistakes

1. Checking the hash map for the complement **after** inserting the current element, causing incorrect self-pairing when duplicates aren't intended to pair with themselves. *Fix*: always check before inserting.
2. Forgetting to normalize keys (case sensitivity, whitespace, canonical signature) when the problem implicitly requires it (e.g., anagram grouping needs a sorted-string or frequency-count key, not the raw string). *Fix*: explicitly design the canonical key before coding.
3. Using a hash map when a hash set would suffice (unnecessary complexity/memory) or vice versa (losing needed associated data). *Fix*: clarify upfront whether you need just membership or an associated value/count.
4. Ignoring the $O(n)$ space cost when the interviewer explicitly asks for $O(1)$ space — missing the cue to pivot to Two Pointers or Cyclic Sort. *Fix*: always state the space trade-off and ask if it's acceptable.
5. Not handling hash collisions or assuming worst-case $O(1)$ always holds — in a systems-design-adjacent follow-up, forgetting to mention amortized analysis and resizing behavior. *Fix*: be ready to discuss load factor and resizing when probed.

Why people fail: hashing feels “easy” so candidates often skip stating the ordering (check-before-insert) and space trade-off considerations explicitly, missing signals that the interviewer is probing for exactly this depth of understanding, especially at Senior/Staff level.

SECTION 16 — Optimization Techniques

- **Time:** Ensure single-pass processing wherever possible (build and query the map in the same loop, not two separate passes, when correctness allows it).
 - **Space:** Use a fixed-size array instead of a general hash map when the key space is small and bounded (e.g., 26 lowercase letters) for better constant factors and cache locality.
 - **Readability:** Name hash structures by their role (`seenIndices`, `charFrequency`), not generic `map/m`.
 - **Interview performance:** Proactively state the space trade-off and the check-before-insert ordering — these two habits alone signal strong hashing fluency.
-

SECTION 17 — Multiple Language Templates

Java

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) return new int[]{seen.get(complement), i};
        seen.put(nums[i], i);
    }
    return new int[]{-1, -1};
}
```

JavaScript

```
function twoSum(nums, target) {
    const seen = new Map();
    for (let i = 0; i < nums.length; i++) {
```

```

        const complement = target - nums[i];
        if (seen.has(complement)) return [seen.get(complement), i];
        seen.set(nums[i], i);
    }
    return [-1, -1];
}

```

PHP

```

function twoSum(array $nums, int $target): array {
    $seen = [];
    foreach ($nums as $i => $num) {
        $complement = $target - $num;
        if (isset($seen[$complement])) return [$seen[$complement], $i];
        $seen[$num] = $i;
    }
    return [-1, -1];
}

```

Python

```

def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return [-1, -1]

```

Go

```

func twoSum(nums []int, target int) []int {
    seen := make(map[int]int)
    for i, num := range nums {
        complement := target - num
        if idx, ok := seen[complement]; ok {
            return []int{idx, i}
        }
        seen[num] = i
    }
    return []int{-1, -1}
}

```

C++

```

vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> seen;
    for (int i = 0; i < (int)nums.size(); i++) {
        int complement = target - nums[i];
        if (seen.count(complement)) return {seen[complement], i};
        seen[nums[i]] = i;
    }
}

```

```
}  
    return {-1, -1};  
}
```

SECTION 18 — Dry Runs

Small Input

```
nums = [2, 7, 11, 15], target = 9  
seen={}  
i=0(2): complement=7, not in seen → seen={2:0}  
i=1(7): complement=2, IN seen (index 0) → return [0,1]
```

Large Input (Conceptual)

For 10^6 elements, hashing processes each element once with $O(1)$ expected map operations — total $\sim 10^6$ operations, versus the $O(10^{12})$ pairwise brute-force alternative, an enormous practical difference.

Corner Case

```
nums = [3,3], target = 6: i=0(3): complement=3, not in seen yet → insert seen={3:0}. i=1(3):  
complement=3, IS in seen (index 0, a different index than current i=1) → return [0,1] correctly,  
since the check-before-insert ordering prevents matching index 1 with itself while still correctly  
matching against the earlier index 0.
```

SECTION 19 — Advanced Concepts

- **Consistent hashing:** used in distributed systems to minimize re-distribution of keys when nodes are added/removed — an advanced, systems-relevant generalization of basic hashing, worth mentioning in Staff-level system design contexts.
 - **Bloom filters:** a probabilistic, space-efficient alternative to hash sets when false positives are acceptable and memory is highly constrained — trades perfect accuracy for massive space savings.
 - **Rolling hash (Rabin-Karp):** computing a hash incrementally as a window slides, enabling $O(1)$ amortized substring/pattern hash updates — a powerful combination of Hashing with Sliding Window for string-matching problems.
 - **Canonical key design:** the art of choosing the right “signature” for grouping (sorted string for anagrams, frequency-count tuple as an alternative avoiding $O(k \log k)$ sort cost, serialized subtree structure for duplicate subtree detection) is often the crux of Medium/Hard hashing problems.
-

SECTION 20 — Senior Engineer Insights

Staff engineers view hashing as the **default tool for “have I seen this” and “how many of this” questions** — but they’re equally quick to recognize its limits: no ordering, $O(n)$ space, and potential worst-case degradation under adversarial input (a real concern in production systems facing untrusted input, mitigated via randomized hash seeds). Interviewers evaluate whether candidates default to hashing appropriately (not over-using it when a simpler $O(1)$ -space alternative like Two Pointers or Cyclic Sort applies) and whether they can discuss the underlying hash table mechanics

(load factor, resizing, collision handling) when probed — signaling genuine understanding versus black-box API usage.

SECTION 21 — Cheat Sheet

PATTERN: Hashing

RECOGNIZE: "have you seen," "frequency," "complement/pair sum" on UNSORTED data, "group by," "duplicate/un

TEMPLATE (complement lookup):

```
seen = {}
for i, x in enumerate(arr):
    if (target - x) in seen: return (seen[target-x], i)
    seen[x] = i
```

COMPLEXITY: $O(n)$ expected time, $O(n)$ space

KEY PROOF: hash function distributes keys ~uniformly, giving $O(1)$ expected bucket size

WATCH FOR: check-before-insert ordering, key normalization, $O(n)$ space trade-off vs $O(1)$ -space alternative

DOESN'T APPLY WHEN: need range/order queries, severe memory constraints

SECTION 22 — Revision Notes (5-Minute Review)

- Hash set = membership only; hash map = membership + associated value/count.
 - Always check-before-insert to avoid incorrect self-pairing.
 - $O(n)$ space trade-off vs $O(1)$ -space Two Pointers (needs sortedness) or Cyclic Sort (needs bounded range).
 - Canonical key design (sorted string, frequency tuple, serialized structure) is the crux of grouping problems.
 - No ordering guarantees — pivot to sorted structures for range/min/max queries.
 - Combine with Prefix Sum (subarray sum counting) or Sliding Window (window state) for advanced problems.
-

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic existence/frequency checks	Two Sum (1), Contains Duplicate (217), Valid Anagram (242)
Intermediate	Grouping and complement counting	Group Anagrams (49), Top K Frequent Elements (347), Intersection of Two Arrays II (350)
Advanced	Cross-pattern combinations	Subarray Sum Equals K (560), Longest Consecutive Sequence (128), Longest Substring Without Repeating Characters (3)
Expert	Multi-structure design	LRU Cache (146), LFU Cache (460), Insert Delete GetRandom $O(1)$ (380)

SECTION 24 — Memory Tricks

- **Mnemonic:** “Seen it? Check the map.” (SCM)
 - **Visualization:** A **coat-check counter** — direct ticket-based retrieval, no rack-searching.
 - **Recognition shortcut:** “Unsorted” + “pair/frequency/duplicate/group” → Hashing, immediately.
-

SECTION 25 — Final Summary

Hashing trades $O(n)$ space for $O(1)$ expected-time existence, frequency, and complement lookups, making it the most broadly applicable pattern for unsorted-data problems that would otherwise require $O(n^2)$ brute force or a costly sort. The single most important thing to remember forever: **always check the map for what you need before inserting the current element, always be explicit about the $O(n)$ space trade-off versus $O(1)$ -space alternatives (Two Pointers, Cyclic Sort), and remember hashing gives you no ordering — the moment a problem needs range/min/max queries over keys, you need a different structure entirely.**